

BhookBuster

From Audit to AI — The Full Journey

What we built, what we fixed, and how we did it.

This document is a complete record of the BhookBuster AI integration project — from the initial codebase audit through security remediation, AI infrastructure design, and the five Codex implementation prompts that shipped three AI features: semantic food search, a personalised recommendation engine, and an AI business analyst dashboard.

4/10

Health score
before we started

3

Critical security
issues fixed

5

Codex prompts
run in order

1

API key
needed (Google)

Phase 1 — Codebase audit

We started by running a full audit of BhookBuster using Codex and a structured prompt that covered three questions: what is this codebase, how is it built, and what can be improved.

What the audit found

Area	Score	Finding
Security	3/10	3 critical issues. exposed internal key, unauthenticated endpoints, open CORS
Performance	5/10	Unbounded queries, regex search, no caching or pagination
Maintainability	4/10	No root workspace, tracked node_modules/dist, missing abstractions
Test coverage	0/10	No unit, integration, or e2e tests found anywhere in the repo
Architecture	5/10	Good service separation but no gateway, no CI/CD, stale JWT claims
Overall health	4/10	Functional product but not safe to extend without security fixes first

How we ran the audit

We built a two-phase prompt system. Phase 1 asked Codex to explore the full file tree and produce a structured report with severity-tagged issues and a scorecard. Phase 2 used the audit findings as hard constraints for designing the AI features — every recommendation referenced real file paths and line numbers from the codebase.

Phase 2 — Security foundation

Before writing a single line of AI code we resolved all three critical findings. Building ML features on top of insecure surfaces amplifies every vulnerability — a vector store full of user data behind an open endpoint is worse than no vector store at all.



Browser-exposed internal key

VITE_INTERNAL_SERVICE_KEY was readable by any user in browser devtools. Replaced the direct internal realtime emit in RiderOrderMap.tsx with an authenticated POST /api/rider/location endpoint — server to server only.



Unauthenticated payment and upload routes

Payment verification and Cloudinary upload had no auth middleware. Added isAuthenticated, ownership checks, file type/size limits, and rate limiting.



Open CORS across all services

Every service and Socket.IO used origin: '*'. Replaced with allowedOrigins array read from CORS_ORIGIN environment variable per deployment.



Broken restaurant ownership checks

Any seller could fetch any restaurant's paid orders. Fixed by joining restaurantId with ownerId === req.user._id before returning data.



Secrets tracked in git

services/auth/.env was committed to the repository. Rotated all credentials, removed from history, added .env.example for every service.



Zero test coverage

No tests existed anywhere. Added Jest + supertest integration tests for auth, ownership, payment idempotency, and analytics scoping as each fix was made.

Phase 3 — AI feature design

With the security foundation in place we designed three AI features. The design prompt was grounded in the audit — every integration point referenced a real file and line number. We also identified shared infrastructure that all three features could reuse.

Semantic food search

3/5

Readiness

Replaces regex search with natural-language understanding. Customers can search 'healthy dinner' or 'spicy chicken under Rs.300' and get results ranked by meaning.

- MenuItems.ts — add embedding fields
- restaurant.ts line 207 — replace regex
- Home.tsx — new DishSearchResultCard

Stack: MongoDB Atlas Vector Search + OpenAI text-embedding-3-small (swapped to Google text-embedding-004)

Personalized recommendations

2/5

Readiness

Recommends dishes and restaurants from order history, favourite cuisines, and behaviour signals. New users see popular nearby; returning users see a personalised For You feed.

- Order.ts + Cart.ts — existing signals
- payment.consumer.ts — trigger profile update
- Home.tsx — For You row

Stack: Embedding similarity + RabbitMQ consumer + UserTasteProfile collection

AI business analyst dashboard

3/5

Readiness

Gives admins and restaurant owners AI-generated narrative insights, anomaly flags, and a natural-language ask-your-data interface scoped by role.

- order.ts line 194 — scoped analytics
- Restaurant.tsx line 114 — insights panel
- Admin.tsx line 50 — platform view

Stack: Mongo aggregation pipelines + Gemini 2.0 Flash narrative layer + pre-approved NL templates

Phase 4 — Keys and infrastructure

We made a key decision: use Google AI (Gemini) instead of OpenAI. Same capabilities, generous free tier, and strong regional infrastructure for an India-based product. This required only one external API key for the entire AI layer.

Google AI API Key aistudio.google.com Free	The only external key needed. Used for text-embedding-004 (768 dimensions) and gemini-2.0-flash for narrative summaries and re-ranking. 1,500 requests/day free.
MongoDB Atlas Vector Search cloud.mongodb.com M10+ cluster	Enabled on the existing Atlas cluster. No new key needed — uses the same MONGODB_URI. Requires M10 or higher tier to support vector indexes.
GATEWAY_HMAC_SECRET Self-generated Free	Not a third-party key. Generated locally with node crypto.randomBytes(32). Shared between ai-gateway and all other services for signing internal requests.
RabbitMQ / Cloudinary / Razorpay / Stripe Already in stack Existing	No changes to existing keys or services. The new USER_EVENT_QUEUE rides the existing RabbitMQ instance.

ai-gateway .env (the only new config file)

```
GOOGLE_AI_API_KEY=AIzaSy... # from aistudio.google.com
GATEWAY_HMAC_SECRET=<64-char-hex> # node -e "require('crypto').randomBytes(32).toString('hex')"
PORT=5010
# Add to every other service .env:
GATEWAY_HMAC_SECRET=
AI_GATEWAY_URL=http://localhost:5010
```

Phase 5 — Codex implementation prompts

We ran five prompts in Codex in strict order. Each prompt assumed the previous one was merged and tested. Codex explored the actual files before writing code, citing line numbers in every change.

1

Security foundation

Prompt 1

Fixed all 6 security issues: CORS, internal key, upload/payment auth, ownership checks, git secrets, and added integration tests for each fix.

2

services/ai-gateway scaffold

Prompt 2

New service with HMAC-signed internal auth, Google Gemini client, and three endpoints: /internal/embed, /internal/insights, /internal/rerank.

3

Embedding pipeline and semantic search

Prompt 3

Schema updates, embedding generation hooks, MongoDB Atlas Vector Search indexes, POST /api/search/semantic endpoint, and DishSearchResultCard in the frontend.

4

Personalized recommendations

Prompt 4

UserTasteProfile + UserFoodEvent collections, event capture endpoint, RabbitMQ recommendations consumer, GET /api/recommendations/home, For You row in UI.

5

AI business analyst dashboard

Prompt 5

Scoped analytics aggregation endpoints, LLM narrative summaries via ai-gateway, ask-your-data with pre-approved templates, RestaurantInsights and PlatformInsights UI.

Phase 6 — Shared AI infrastructure

A key insight from the design phase: all three AI features share the same underlying components. Building this once avoided duplicate model calls, scattered API keys, and inconsistent rate limiting across services.

New service. Owns all model API calls, keys, rate limits, retries, prompts, and is the only service reachable from the frontend. Uses HMAC-SHA256 signed requests instead of JWT.

Menu items, restaurants, and user search queries all embed through a shared service using OpenAI's text-embedding-004 (768 dimensions). Regenerated automatically on menu updates.

Vectors live in the same MongoDB cluster already used by Mongoose. Uses a shared index on restaurants.embedding, both 768-dimensional cosine. Supports hybrid search (keyword + radius).

New queue on the existing RabbitMQ instance. Carries search, click, cart, and recommendation events. The recommendations consumer listens and updates UserTasteProfile.

Phase 7 — Build order and verification

We followed a strict 8-step build order where each step unblocked the next. After all five Codex prompts ran, we verified the implementation with curl commands executed directly in the Codex session.

1	Security foundation CORS, upload/payment auth, internal key removal, ownership fixes, test harness.
2	services/ai-gateway Internal model service. HMAC auth, Google Gemini, rate limits, logging.
3	Embedding pipeline Schema updates, embedding hooks on menu/restaurant create/update, Atlas indexes.
4	Semantic food search MVP POST /api/search/semantic. Hybrid vector + metadata filter. Home UI cards.
5	User event stream + consumer USER_EVENT_QUEUE, UserTasteProfile, recommendations consumer on PAYMENT_SUCCESS.
6	Personalized recommendations GET /api/recommendations/home. Content-based. For You row in frontend.
7	Scoped analytics endpoints Owner-scoped Mongo aggregations. Revenue, peak hours, top dishes, customer rate.
8	AI business analyst dashboard LLM summaries, anomaly detection, ask-your-data templates, insights UI panels.

Verification commands run in Codex

After all prompts completed, we verified the implementation by asking Codex to run these checks and fix any failures inline before moving on.

- ✓ ai-gateway starts and /health responds
- ✓ HMAC-signed /internal/embed returns 768-dimensional vector for a food query
- ✓ Vector Search indexes created on menuitems and restaurants collections
- ✓ POST /api/search/semantic returns ranked dish results for 'healthy dinner'
- ✓ All Jest integration tests pass across workspaces

Next step: for each AI feature, run a targeted follow-up in Codex — 'Implement feature N, MVP scope, with integration tests' — to keep implementation grounded in the file-level analysis rather than generic boilerplate.